What MVTec AD Can and Cannot Tell Us About Deployment: Operating Points, Alarm Budgets and Cost-Calibrated Thresholds for Unsupervised Industrial Anomaly Detection

Utkarsh Sengar
sengarutkarsh@gmail.com

November 2, 2026

Abstract

Image AUROC and AU-PRO rank anomaly detectors without fixing a decision threshold, yet an inspection line runs at one threshold, under an operator alarm budget, with missed defects far costlier than false alarms. We evaluate PatchCore-style, PaDiM-style and autoencoder detectors on 7 MVTec AD categories (63 runs, 3 seeds) at fixed operating points. Thresholds are calibrated only on held-out nominal training images, and every realised rate is reported with an exact interval and with the resolution the test set allows. Three findings follow. First, MVTec AD test sets contain 20–58 nominal images, so a budget of 5 false alarms per 1,000 parts cannot be verified: the coarsest category resolves only 50 per 1,000. Second, thresholds calibrated on held-out training images transfer poorly: the realised test false-alarm rate exceeds the 5/1k target in 60% of run–category evaluations. Third, cost-calibrated thresholding (CCT), i.e. constrained empirical risk minimisation using a few labelled defects, changes expected cost by -51% (PatchCore), -10% (PaDiM) and -10% (autoencoder) relative to a nominal-only budget threshold out of sample, differences that categories-as-units tests cannot distinguish from zero. Code, score archives and every table are generated by a single pipeline.

1 Introduction

Benchmarks for unsupervised visual anomaly detection report threshold-free ranking metrics: image-level AUROC and the per-region-overlap curve integrated up to a 30% false-positive rate (AU-PRO) [1–3]. A deployed inspector instead commits to a threshold. Two constraints then dominate: human reviewers can absorb only a few false alarms per thousand parts, and an escaped defect costs many times more than a false alarm [8]. This paper asks which statements about such operating points the standard MVTec AD protocol can support.

We make three contributions. (i) A threshold protocol that never touches the test set: thresholds are calibrated on a held-out 20% of the nominal training images, and every realised test rate is reported with a Clopper–Pearson interval [4] and with the resolution the test set permits. (ii) An out-of-sample evaluation of cost-calibrated thresholding with repeated stratified splits. (iii) Statistical comparisons that treat categories, not seeds, as the independent units [6], which makes explicit how little significance seven categories can carry.

2 Setup

Data. We use 7 MVTec AD categories (bottle, cable, carpet, grid, hazelnut, leather, metal nut). Every category is checked against the official per-split, per-defect-type image counts before use. Images are resized

to 256×256. For each seed, 20% of `train/good` is held out as a nominal calibration set (42–78 images per category); models are fitted on the rest.

Detectors. We reimplement PatchCore [11] and PaDiM [5] with an ImageNet ResNet-18 backbone, and train a convolutional autoencoder from scratch. PatchCore keeps a 10% memory bank selected by exact greedy k-center [12] in a random 128-d projection; PaDiM fits per-location Gaussians on 100 random channels. All three produce a pixel map, smoothed with $\sigma = 4$, and use its maximum as the image score. The seed changes the calibration split, PatchCore's projection and start point, PaDiM's channel subset and the autoencoder's initialisation. These are reimplementations under one protocol, not reproductions of the published numbers.

Metrics. AU-PRO is computed exactly at every distinct score following the MVTec protocol. For robustness we apply 6 corruption types at 3 severities (18 conditions) after resizing, in the spirit of [9], and report the non-negative mean robustness degradation (MRD) and the signed mean change (MPC). A part is flagged when its score is at least τ. With false-alarm cost 1, missed-defect cost r and production defect rate π, the expected cost per part is

$$C(\tau) = (1 - \pi)\,\mathrm{FPR}(\tau) + \pi\, r\,\mathrm{FNR}(\tau). \tag{1}$$

We use $\pi = 0.01$ and $r = 10$ unless stated otherwise, and this is the only cost definition in the paper.

Thresholds. The *nominal-only budget* rule picks the lowest τ whose false-positive rate on the calibration nominals is at most $b/1000$. *CCT* minimises $C(\tau)$ on labelled calibration data, optionally subject to the same budget. Because CCT needs defects, it is evaluated with 20 stratified 50/50 splits of each run's test set: thresholds use one half plus the held-out nominals; metrics use the other half. An *oracle* that picks the cost-optimal threshold on the evaluation half itself gives a lower bound.

Statistics. Per-category intervals use a two-stage bootstrap over seeds and test images ($B = 2000$) [7]. Method and rule comparisons use two-sided Wilcoxon signed-rank tests [13] with one value per category, adjusted with Holm's procedure [10]. With n categories the smallest attainable p-value is $2/2^n$.

3 Results

Ranking metrics. Table 1 reproduces the familiar ordering: PatchCore-style (mean image AUROC 0.992, AU-PRO 0.928) ahead of PaDiM-style (0.930, 0.914) and the autoencoder (0.647, 0.590). Table 2 and Fig. 2 show that the feature-based detectors lose most under noise, motion blur and downscaling. The autoencoder's small degradation reflects its weak clean performance rather than robustness.

Operating points. Table 3 and Fig. 1 report the operating point at a target of 5 false alarms per 1,000 nominal parts. Two effects limit what can be claimed. The test sets are too small to observe rates of this size: one false alarm on the smallest nominal set already amounts to 50 per 1,000. And the calibration images do not represent the nominal test images well, so realised test false-alarm rates exceed the target in 60% of evaluations (Fig. 1, centre). A threshold that satisfies an alarm budget on training-distribution nominals is therefore no guarantee on the test distribution, and per-mille budgets need far larger nominal validation sets than MVTec AD provides.

Table 1: Clean test results on 7 MVTec AD categories. Per category: mean ± std over 3 seeds (seeds change the train/calibration split, PatchCore's projection and start point, PaDiM's channel subset and the autoencoder's initialisation). *Mean* rows: mean ± std over categories. AU-PRO is integrated up to FPR $= 0.3$ with the exact MVTec protocol.

Category	Method	Image AUROC	Pixel AUROC	AU-PRO
Bottle	PatchCore	1.000 ± 0.000	0.983 ± 0.000	0.933 ± 0.001
Bottle	PaDiM	0.997 ± 0.001	0.984 ± 0.001	0.950 ± 0.001
Bottle	ConvAutoencoder	0.784 ± 0.008	0.735 ± 0.004	0.530 ± 0.003
Cable	PatchCore	0.991 ± 0.002	0.985 ± 0.000	0.940 ± 0.002
Cable	PaDiM	0.855 ± 0.008	0.971 ± 0.003	0.905 ± 0.005
Cable	ConvAutoencoder	0.336 ± 0.008	0.617 ± 0.055	0.406 ± 0.010
Carpet	PatchCore	0.986 ± 0.000	0.990 ± 0.000	0.935 ± 0.002
Carpet	PaDiM	0.964 ± 0.005	0.987 ± 0.000	0.943 ± 0.004
Carpet	ConvAutoencoder	0.335 ± 0.025	0.735 ± 0.020	0.417 ± 0.013
Grid	PatchCore	0.968 ± 0.004	0.977 ± 0.002	0.902 ± 0.005
Grid	PaDiM	0.862 ± 0.050	0.926 ± 0.011	0.832 ± 0.050
Grid	ConvAutoencoder	0.850 ± 0.044	0.818 ± 0.026	0.620 ± 0.040
Hazelnut	PatchCore	1.000 ± 0.000	0.987 ± 0.000	0.881 ± 0.005
Hazelnut	PaDiM	0.858 ± 0.010	0.981 ± 0.002	0.869 ± 0.003
Hazelnut	ConvAutoencoder	0.981 ± 0.003	0.978 ± 0.003	0.874 ± 0.013
Leather	PatchCore	1.000 ± 0.000	0.992 ± 0.000	0.973 ± 0.000
Leather	PaDiM	0.999 ± 0.001	0.994 ± 0.001	0.982 ± 0.004
Leather	ConvAutoencoder	0.860 ± 0.005	0.968 ± 0.007	0.933 ± 0.005
Metal Nut	PatchCore	0.997 ± 0.001	0.984 ± 0.000	0.935 ± 0.001
Metal Nut	PaDiM	0.973 ± 0.004	0.972 ± 0.005	0.916 ± 0.006
Metal Nut	ConvAutoencoder	0.383 ± 0.012	0.671 ± 0.009	0.349 ± 0.022
Mean	PatchCore	0.992 ± 0.012	0.985 ± 0.005	0.928 ± 0.029
Mean	PaDiM	0.930 ± 0.068	0.974 ± 0.022	0.914 ± 0.051
Mean	ConvAutoencoder	0.647 ± 0.283	0.789 ± 0.140	0.590 ± 0.233

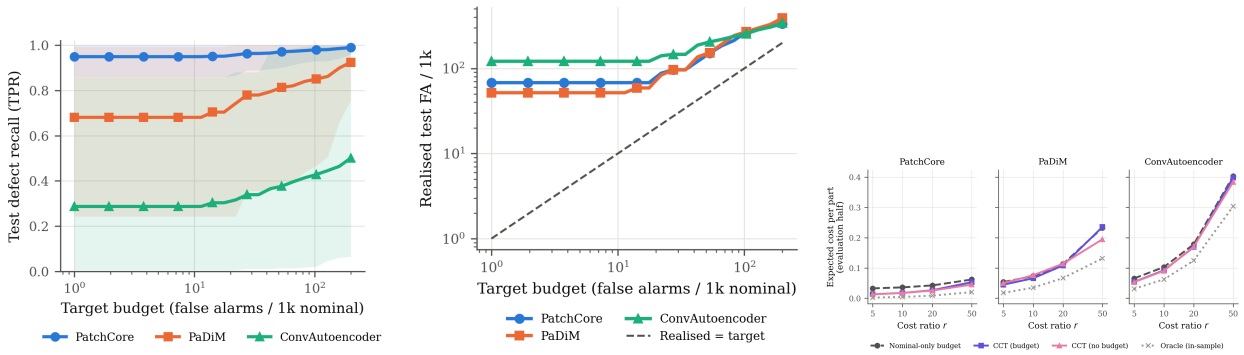


Figure 1: Left: test defect recall when the threshold is calibrated on held-out nominal training images for a target budget (macro-average over categories; band: range over categories). Centre: realised test false-alarm rate against the target; the dashed line marks equality. Rates below the test-set resolution cannot be observed. Right: out-of-sample expected cost of each threshold rule as the cost ratio grows.

Table 2: Robustness over 18 corruption conditions (6 types $\times$ 3 severities, applied after resizing to 256 px). MRD $= \frac{1}{18}\sum_i \max(0, M_{clean} - M_i)$; MPC is the signed mean. Values are means over 3 seeds.

Category	Method	MRD (AUROC) ↓	MRD (AU-PRO) ↓	MPC (AUROC)
Bottle	PatchCore	0.057	0.108	+0.057
Bottle	PaDiM	0.179	0.234	+0.179
Bottle	ConvAutoencoder	0.115	0.208	+0.105
Cable	PatchCore	0.171	0.144	+0.171
Cable	PaDiM	0.123	0.195	+0.119
Cable	ConvAutoencoder	0.001	0.068	−0.075
Carpet	PatchCore	0.106	0.146	+0.106
Carpet	PaDiM	0.248	0.279	+0.247
Carpet	ConvAutoencoder	0.005	0.085	−0.121
Grid	PatchCore	0.327	0.485	+0.327
Grid	PaDiM	0.295	0.390	+0.295
Grid	ConvAutoencoder	0.244	0.133	+0.243
Hazelnut	PatchCore	0.049	0.073	+0.049
Hazelnut	PaDiM	0.102	0.217	+0.078
Hazelnut	ConvAutoencoder	0.044	0.095	+0.043
Leather	PatchCore	0.056	0.067	+0.056
Leather	PaDiM	0.143	0.099	+0.143
Leather	ConvAutoencoder	0.060	0.100	+0.053
Metal Nut	PatchCore	0.168	0.190	+0.168
Metal Nut	PaDiM	0.224	0.345	+0.222
Metal Nut	ConvAutoencoder	0.003	0.031	−0.092
Mean	PatchCore	0.133	0.173	+0.133
Mean	PaDiM	0.188	0.251	+0.183
Mean	ConvAutoencoder	0.067	0.103	+0.022

Cost-calibrated thresholds. Table 4 compares threshold rules out of sample. Using labelled calibration defects changes the expected cost by -51%, -10% and -10% for the three detectors. Budget violations on the evaluation halves occur for every rule (nominal-only: 28%, 38%, 26%; CCT with budget: 10%, 29%, 18%). Every deployable rule stays well above the cost of the in-sample oracle, which suggests that the main limitation is the amount of calibration data rather than the choice of rule. With 7 categories as units, none of the rule comparisons is significant after Holm adjustment (Table 4); the same holds for the method comparisons in Table 5, including PatchCore versus the autoencoder.

Latency. Table 6 reports batch-1 latency on the GPU used for the systems measurements. End-to-end time, which includes decoding and resizing the full-resolution PNG, is several times the model time for all three detectors, so image I/O rather than the model dominates throughput at batch size 1.

Coreset selection. PatchCore's memory bank is selected with exact greedy k-center. Running the same greedy rule on the GPU is 8$\times$ faster than a vectorised NumPy implementation at $N = 100,000$ patches (Table 7, Fig. 3) and reaches the same coverage radius; the selected indices can diverge after float32 rounding breaks a near-tie differently (92.8% identical at that size). Adding the 50 farthest points per step is much faster but gives a larger coverage radius, most visibly for small N.

Table 3: Operating point at a target budget of 5 false alarms per 1,000 nominal parts. The threshold is set on held-out train/good images only (42–78 per category) and applied to the full test set. Brackets: 95% two-stage bootstrap intervals over seeds and test images ($B = 2000$). "Resolution" is the smallest non-zero rate the test set can express (1000 / nominal test images). Expected cost uses defect prior $\pi = 0.01$ and cost ratio $r = 10$.

Category	Method	Test TPR	Test FA/1k	Resolution/1k	Exp. cost
Bottle	PatchCore	1.000 [1.000, 1.000]	0.0 [0.0, 0.0]	50.0	0.0000
Bottle	PaDiM	0.937 [0.889, 0.974]	33.3 [0.0, 100.0]	50.0	0.0393
Bottle	ConvAutoencoder	0.444 [0.296, 0.603]	16.7 [0.0, 66.7]	50.0	0.0721
Cable	PatchCore	0.866 [0.797, 0.924]	0.0 [0.0, 0.0]	17.2	0.0134
Cable	PaDiM	0.243 [0.134, 0.388]	23.0 [0.0, 63.2]	17.2	0.0985
Cable	ConvAutoencoder	0.007 [0.000, 0.022]	34.5 [0.0, 86.2]	17.2	0.1334
Carpet	PatchCore	0.978 [0.955, 0.996]	309.5 [154.8, 500.3]	35.7	0.3087
Carpet	PaDiM	0.835 [0.738, 0.925]	59.5 [0.0, 142.9]	35.7	0.0754
Carpet	ConvAutoencoder	0.079 [0.030, 0.131]	190.5 [71.4, 321.4]	35.7	0.2807
Grid	PatchCore	0.860 [0.801, 0.912]	47.6 [0.0, 111.1]	47.6	0.0612
Grid	PaDiM	0.316 [0.193, 0.450]	15.9 [0.0, 63.5]	47.6	0.0841
Grid	ConvAutoencoder	0.006 [0.000, 0.023]	0.0 [0.0, 0.0]	47.6	0.0994
Hazelnut	PatchCore	0.967 [0.938, 0.990]	0.0 [0.0, 0.0]	25.0	0.0033
Hazelnut	PaDiM	0.533 [0.352, 0.781]	108.3 [33.3, 208.3]	25.0	0.1539
Hazelnut	ConvAutoencoder	0.586 [0.376, 0.738]	0.0 [0.0, 0.0]	25.0	0.0414
Leather	PatchCore	1.000 [1.000, 1.000]	72.9 [20.8, 135.4]	31.2	0.0722
Leather	PaDiM	0.993 [0.978, 1.000]	31.2 [0.0, 83.3]	31.2	0.0317
Leather	ConvAutoencoder	0.859 [0.808, 0.906]	500.0 [375.0, 614.6]	31.2	0.5091
Metal Nut	PatchCore	0.975 [0.953, 0.993]	45.5 [0.0, 121.2]	45.5	0.0475
Metal Nut	PaDiM	0.910 [0.864, 0.957]	90.9 [15.2, 197.0]	45.5	0.0990
Metal Nut	ConvAutoencoder	0.029 [0.011, 0.050]	106.1 [30.3, 197.0]	45.5	0.2021

Table 4: Out-of-sample threshold selection, cost ratio $r = 10$, prior $\pi = 0.01$, budget 5/1k. Each run's test set is split 50/50 (stratified) 20 times; thresholds use the calibration half plus held-out train/good nominals; metrics are measured on the other half. Cost: mean $\pm$ std over categories (each category averaged over seeds and splits). "Violations": share of evaluations whose realised FA/1k exceeds the budget. p: Holm-adjusted two-sided Wilcoxon signed-rank test against the nominal-only budget rule with categories as units ($n = 7$, smallest attainable $p = 0.016$ before adjustment).

Method	Rule	Expected cost	FA/1k	Violations	p (Holm)
PatchCore	Nominal-only budget	0.0359 ± 0.0300	29.7	28%	
PatchCore	CCT (budget)	0.0175 ± 0.0231	8.6	10%	1.000
PatchCore	CCT (no budget)	0.0173 ± 0.0227	8.6	10%	1.000
PatchCore	Oracle (in-sample)	0.0043 ± 0.0062	0.0	0%	
PaDiM	Nominal-only budget	0.0733 ± 0.0335	33.9	38%	
PaDiM	CCT (budget)	0.0660 ± 0.0433	23.9	29%	0.875
PaDiM	CCT (no budget)	0.0761 ± 0.0566	43.5	40%	1.000
PaDiM	Oracle (in-sample)	0.0350 ± 0.0334	1.4	3%	
ConvAutoencoder	Nominal-only budget	0.1031 ± 0.0402	28.2	26%	
ConvAutoencoder	CCT (budget)	0.0922 ± 0.0286	15.8	18%	0.781
ConvAutoencoder	CCT (no budget)	0.0906 ± 0.0317	16.4	18%	0.188
ConvAutoencoder	Oracle (in-sample)	0.0624 ± 0.0356	0.0	0%	

Table 5: Paired comparison of expected cost ($r = 10$, budget 5/1k) between methods, one value per category (mean over seeds). Two-sided Wilcoxon signed-rank test; Holm adjustment over the three comparisons.

Comparison	n	**Hodges–Lehmann** Δ	p	p **(Holm)**
ConvAutoencoder vs PaDiM	7	+0.0685	0.156	0.312
ConvAutoencoder vs PatchCore	7	+0.0876	0.031	0.094
PaDiM vs PatchCore	7	+0.0342	0.578	0.578

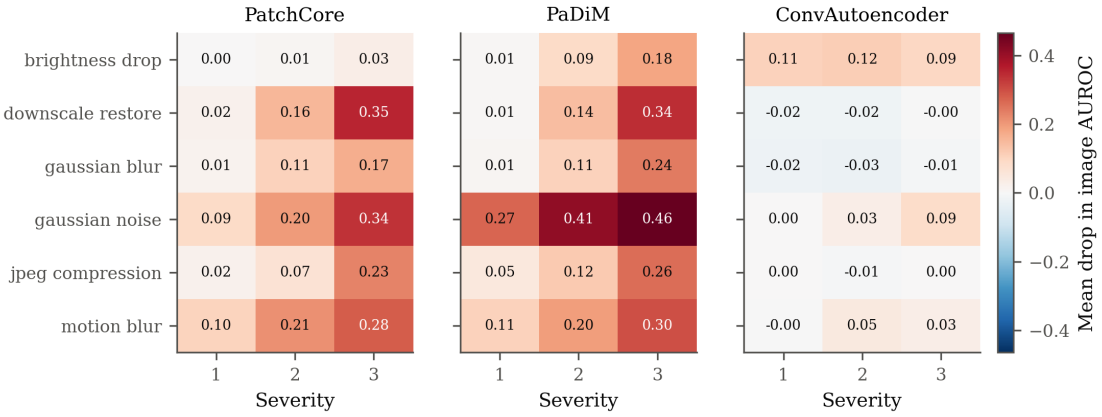


Figure 2: Mean drop in image AUROC per corruption and severity (averaged over categories and seeds).

Table 6: Latency at batch size 1 on NVIDIA GeForce RTX 4050 Laptop GPU, mean over 7 categories. T_{model}: device tensor to scores and maps; T_{e2e}: PNG on disk to scores and maps.

Method	**P50** T_{model} **(ms)**	**P95** T_{model} **(ms)**	**P50** T_{e2e} **(ms)**	**Peak VRAM (MB)**
PatchCore	12.08	13.94	43.18	197
PaDiM	7.65	9.87	32.37	219
ConvAutoencoder	4.87	6.18	30.90	11

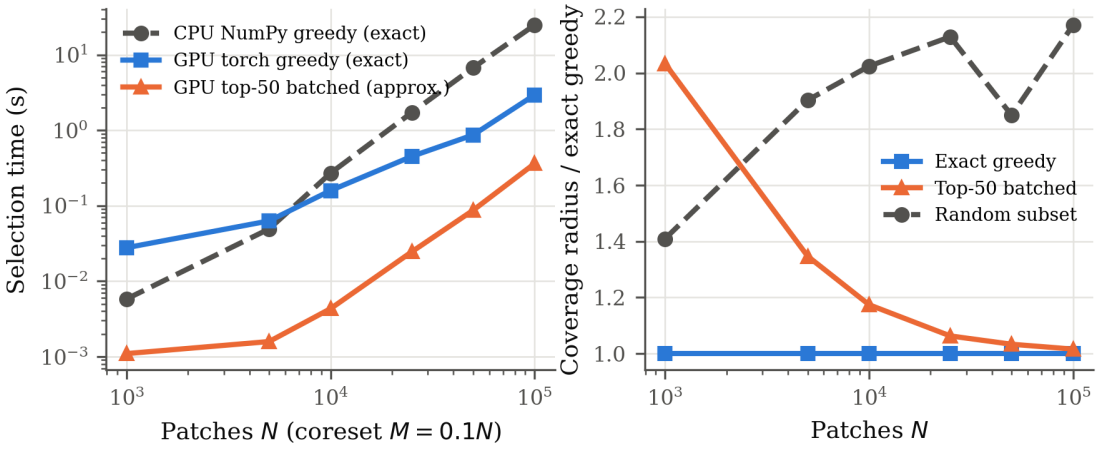


Figure 3: Greedy k-center selection time and coverage radius relative to exact greedy ($D = 128$, $M = 0.1N$).

Table 7: Greedy k-center coreset selection ($M = 0.1N$) on real PatchCore features, all cells measured on NVIDIA GeForce RTX 4050 Laptop GPU (median of 3 timings). Agreement: share of selected indices identical to the NumPy reference; float32 rounding can break near-ties differently, after which the greedy sequences diverge. Radius columns: coverage radius (k-center objective) of the GPU greedy relative to the NumPy greedy, and of the approximate top-50 batched variant relative to exact greedy.

N	D	CPU (s)	GPU (s)	Speedup	Agreement	GPU/CPU radius	Top-50/greedy radius
1,000	64	0.00	0.047	0.0×	100.0%	1.000	2.070
5,000	64	0.19	0.118	1.6×	100.0%	1.000	1.467
10,000	64	0.14	0.168	0.8×	99.8%	1.000	1.175
25,000	64	1.08	0.449	2.4×	98.9%	1.000	1.063
50,000	64	6.22	0.828	7.5×	97.9%	1.000	1.039
100,000	64	16.53	1.706	9.7×	52.5%	1.000	1.019
1,000	128	0.01	0.028	0.2×	100.0%	1.000	2.033
5,000	128	0.05	0.063	0.8×	100.0%	1.000	1.347
10,000	128	0.27	0.159	1.7×	100.0%	1.000	1.174
25,000	128	1.71	0.450	3.8×	99.3%	1.000	1.063
50,000	128	6.74	0.866	7.8×	96.9%	1.000	1.033
100,000	128	24.71	2.933	8.4×	92.8%	1.000	1.016
1,000	256	0.00	0.031	0.1×	100.0%	1.000	1.912
5,000	256	0.08	0.064	1.3×	100.0%	1.000	1.391
10,000	256	0.57	0.190	3.0×	99.8%	1.000	1.163
25,000	256	3.70	0.400	9.3×	97.9%	1.000	1.060
50,000	256	13.01	1.455	8.9×	96.3%	1.000	1.037
100,000	256	41.94	5.244	8.0×	9.6%	1.000	1.014

4 Limitations

The baselines use ResNet-18 at 256 px and do not match published PatchCore and PaDiM numbers. Seven categories bound the strength of any between-method test. The corruptions are synthetic proxies. Operator-load projections in the accompanying report resample the measured scores and add no information beyond them. The CCT evaluation draws its labelled defects from the test set, which a real line would have to collect separately. Latency and the GPU coreset-selection benchmark are reported only when measured on a GPU.

5 Conclusion

At deployment-relevant operating points, MVTec AD supports ranking statements but not alarm-budget guarantees: its nominal test sets are too small to resolve per-mille rates, and nominal training images do not calibrate thresholds for the test distribution. Cost-calibrated thresholds are cheap to compute and easy to evaluate honestly, but on this benchmark their benefit cannot be separated from noise. We release the full pipeline so that each of these statements can be checked and extended to larger nominal validation sets.

References

[1] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9592–9600, 2019.

[2] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. Uninformed students: Student–teacher

anomaly detection with discriminative latent embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4183–4192, 2020.

[3] Paul Bergmann, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger. The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision*, 129(4):1038–1059, 2021.

[4] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[5] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. PaDiM: A patch distribution modeling framework for anomaly detection and localization. In *Pattern Recognition. ICPR International Workshops and Challenges*, volume 12664 of *Lecture Notes in Computer Science*, pages 475–489. Springer, 2021.

[6] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[8] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978, 2001.

[9] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019.

[10] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2): 65–70, 1979.

[11] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14318–14328, 2022.

[12] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations (ICLR)*, 2018.

[13] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.